



DEEP LEARNING INDABA 2026 • TUTORIAL

Introduction to Agentic AI

Building Conversational Agents with Persistent Memory

Theme: Sovereign Intelligence, Africa's Path in a Frontier AI World

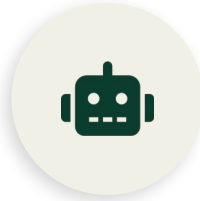
David Agbolade • 2-hour in-person tutorial, lecture and live coding

Why agents, and why now?



Yesterday: chatbots

Answer one question, then forget. Each reply starts from nothing.



Today: agents

Hold a conversation, use tools, remember, and act over time toward a goal.



The missing piece

Memory is what turns a clever reply into a useful, personal assistant.

The sovereign-intelligence lens

As frontier AI concentrates abroad, three questions decide whether the agents serving African users are truly ours:



Who builds them?

Local engineers solving locally-relevant problems, not imported one-size-fits-all tools.



Who owns them?

Ownership of the data, the models, and the product roadmap.



Who hosts them?

Where the model runs, where user data lives, and who pays the bill.

Our two hours together

0	Welcome & framing	5 min
1	What is an AI agent?	20 min
2	Memory in conversational AI	20 min
3	Building an agent in Python (live)	20 min
4	Giving agents tools (live)	15 min
5	Persistent memory with embeddings (live)	20 min
6	Talking to our griot (demo)	10 min
7	Sovereign intelligence	5 min
8	Q&A and resources	5 min



“Today we build a simplified version of the same concepts that run in real production systems.”

Small, honest, and runnable, then connected to systems that are actually deployed.

SEGMENT 1

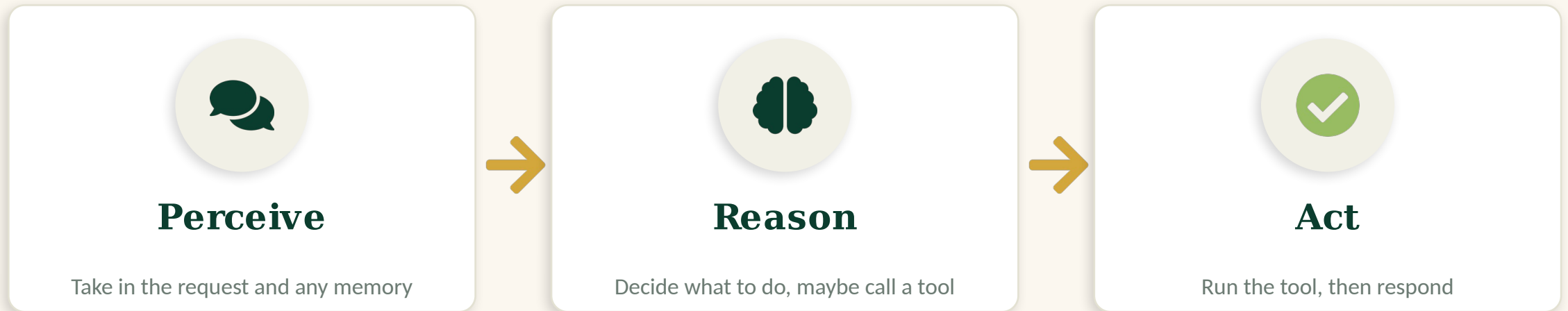
What is an AI agent?

20 min



From one call to an agent

A single language-model call maps text in to text out. An agent wraps that call in a loop:



The loop is what makes it an agent, not the model alone.

Agent vs. chatbot vs. plain prompting

	Plain prompt	Chatbot	Agent
Remembers context	No	This session	Short and long-term
Can use tools	No	Rarely	Yes, central
Acts toward a goal	No	No	Yes, over many steps
Survives a restart	No	No	Yes, with persistent memory

Anatomy of an agent



Model

The reasoning engine, for example a small open model.



Instructions

The system prompt: who it is and how to behave.



Tools

Functions it can call to fetch info or take action.



Memory

What it carries across turns and across sessions.

Our running example: a griot

We build a griot, a traditional oral storyteller, that co-weaves an original, evolving folktale with the room.

It is a fully standalone project: clone it, run it, extend it. Nothing is tied to any product.



Why a storyteller?

A good tale must remember its own characters and events, across turns and across sittings. Here, memory is not an add-on; it is the whole point.

SEGMENT 2

Memory in conversational AI

20 min



Models are stateless

A language model only knows what is inside the current request. To “remember” a conversation, we resend the whole history every single turn.



No hidden state

The model does not hold the chat; your code does.



History is resent

Everything it should know must travel in the prompt.



So memory is our job

We decide what to keep, compress, and recall.

The context window

Big, but not infinite

Every model can only read so many tokens at once. A long conversation, or a long document, eventually overflows it, and the oldest content has to be dropped.

And you pay per token

- Resending a growing history costs money on every turn.
- On tight budgets and low bandwidth, that cost is not optional to manage.
- So we compress, and we retrieve only what is relevant.

Short-term vs. long-term memory



Short-term

- Lives only in the current session
- The running conversation history
- Bounded by the context window
- Cleared on restart



Long-term

- Persists across sessions
- Durable facts: characters, world, preferences
- Stored outside the model in a vector store
- Retrieved by meaning, on demand

What to remember, and what to forget



Time-to-live (TTL)

Let transient chatter expire; keep the durable facts.



Summarise

Collapse old turns into a short “story so far.”



Cost

Every remembered token is a token you resend and pay for.

Why memory matters more here



On low-bandwidth, intermittent connections, sessions drop, often without warning.

Persistent memory lets an agent pick up exactly where a user left off, without making them repeat themselves or resend a long history over an expensive, fragile link.

Good memory design is also good cost and reliability design.

LIVE CODING

```
notebooks/01_short_term_memory.ipynb
```

Goal: see memory as a plain list, watch it overflow, then compress it.

- Keep the whole conversation in a Python list and resend it each turn
- Count the real tokens and watch the number climb toward the limit
- Feel the cost: every turn resends everything
- Compress the old turns into a short story-so-far summary
- See the token count drop while the memory stays intact

SEGMENT 3

Building an agent in Python

20 min



LIVE CODING

`notebooks/00_minimal_agent.ipynb`

Goal: a working griot agent on a small open model, no API key, in under 100 lines.

- Load a small open model, then one call: text in, text out
- Add a tool the griot can call (a fitting saying)
- Let the model decide when to use it (reason, then act)
- Add in-session memory so characters persist across turns
- Watch it forget on restart, which motivates Segment 4

What we just built



We have

- A loop: perceive, reason, act
- A tool the model calls on its own
- Short-term memory within a session



But it forgets

- Close the notebook and the whole tale is gone
- History grows every turn and fills the window
- We need memory that survives restarts

SEGMENT 4

Giving agents tools

15 min



Why an agent needs tools

On its own, a model cannot reach the live world; it is frozen at training time. Tools let it call real services and use the result.



Frozen in time

It cannot know anything happening now, like today's weather.



No live data

On its own it cannot reach the internet or any live service.



Tools bridge the gap

A tool is a function the agent calls to fetch, compute, or act, then it uses the result.

LIVE CODING

`notebooks/04_agent_tools.ipynb`

Goal: let the agent decide to call a tool, then use the result.

- Ask the model the weather in Nairobi right now: it cannot know
- Give it a tool that calls a free weather API (no key)
- Let the agent reason, act, respond using the real data
- The reply now shows the true, current temperature, fetched live
- The point: tools give agents live data, search, code, and actions

SEGMENT 5

Persistent memory with embeddings

20 min



Embeddings, intuitively

An embedding turns text into a vector, a list of numbers that captures meaning. Similar meaning gives vectors that point the same way, even with no shared words.

Near in meaning, near in vector space

“The hunter found a glowing stone by the river”

“Near the water, Ado discovered a shining rock”



0.78

high similarity

“The market was crowded at the festival”



0.06

low similarity

Cosine similarity

Compares the direction of two vectors. 1.0 means the same meaning, 0 means unrelated. This single number powers semantic search.

LIVE CODING

```
notebooks/02_vector_memory.ipynb
```

Goal: a saga that survives a restart and is recalled by meaning.

- Embed story events into vectors
- Store them in a 25-line vector memory
- Retrieve the relevant ones by meaning
- Augment the prompt, then continue the tale
- Reload from disk, and the saga persists

From prototype to production

Concept	Your prototype	In production
Embeddings	text-embedding-004	A managed embedding model
The store	Python list and cosine loop	Vector DB with an HNSW index
Persistence	A JSON file on disk	Short-term TTL plus permanent store
Retrieval	Top-k by cosine	Parallel context assembly
Scale	One saga, a few facts	Thousands of items, many users

The same core moves everywhere: embed, store, retrieve by similarity, then augment the prompt.

SEGMENT 6

Talking to our griot

10 min



An open conversation

We put the notebook down and just talk to the agent. The room suggests where the tale should go, and we watch the griot keep the characters and the thread consistent, live.

US Begin a tale about a young weaver named Ada who finds a strange seed.

GRIOT In the village by the river, Ada planted the seed she could not name.

US A drought comes. What does she do?

GRIOT Remembering the seed's promise, Ada carried it to the dry riverbed.

We never repeat Ada's name; the agent remembers it within the conversation.

It remembers after a restart



Restart the kernel

Short-term memory is gone: the running conversation list is empty. A plain chatbot would start from nothing.



Ask: what did she promise?

The agent retrieves the answer from its vector memory on disk and carries on. The saga survived, because long-term memory is persistent.

Running open models: honest trade-offs



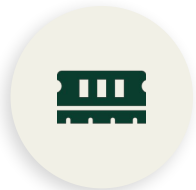
No key, model stays local

The model runs privately; the only calls out are the tools you choose to add.



Size vs. speed

A tiny model is fast and runs anywhere; a bigger one writes better but wants a GPU.



Good enough to teach

The point is the loop and the memory, not poetry. A small model shows both clearly.



Yours to scale

Swap in a larger open model or a real vector DB later, on your own infrastructure.

SEGMENT 7

Sovereign intelligence

5 min



Challenges, and the path forward



Data and bias

- Training on non-African data
- Models that miss local context
- Trust gaps in high-stakes use



Sovereignty

- Where models run
- Where user data lives
- Who pays the API bill



Path forward

- Local datasets and languages
- Local or self-hosted models
- Own your hosting and stack

Take it with you



Tutorial repo

Three runnable notebooks (griot agent) plus the slides. Self-contained, open models, no API key.



Open models used

Qwen2.5-0.5B-Instruct for text, all-MiniLM-L6-v2 for embeddings. Both open and local.



Go further

Swap the list for a real vector DB (Redis, Qdrant, pgvector); add a short and long-term split.



Make it yours

Try a larger open model and self-host the whole stack on your own infrastructure.



Questions & discussion

Build something locally-owned. Let's talk about how.

David Agbolade · dagbolade72@gmail.com